

AI Red-Team vs Blue-Team Simulator

HACK IN HILLS '26 · PROBLEM STATEMENT 04: BAYORA

A simple web-based AI security testing prototype

TEAM VURVE · MEMBER NAMES & ROLES

[Jigyasa Singh]
Team Lead

[Ram Kumar]
Team Member

[Nitin Kumar]
Team Member

PROBLEM STATEMENT & THEME

Problem

- AI systems need security testing before deployment
- Bayora uses a Red Team to test AI systems and a Blue Team to defend them
- Preventing information leakage is a core challenge
- Trustworthy audit records must be maintained

Impact

- Demonstrates how adversarial AI testing can work
- Makes security findings easier to understand
- Provides a clear record of testing activity

Theme Alignment

- Hack in Hills ’26 – Problem Statement 04: Bayora
- Red-Team attacks tested against Blue-Team defenses
- Risk scoring and auditability built in

SOLUTION

OVERVIEW

A web application that simulates Red-Team attacks and Blue-Team defenses. The system analyzes each test, assigns a risk level, and records the result.

KEY FEATURES

1. Red-Team panel to run security tests
2. Blue-Team panel to detect and block suspicious tests
3. **Risk analyzer: Low / Medium / High**
4. Live dashboard with attack feed and reports
5. Hash-chained audit log, alerts and real sign-in

UNIQUE VALUE

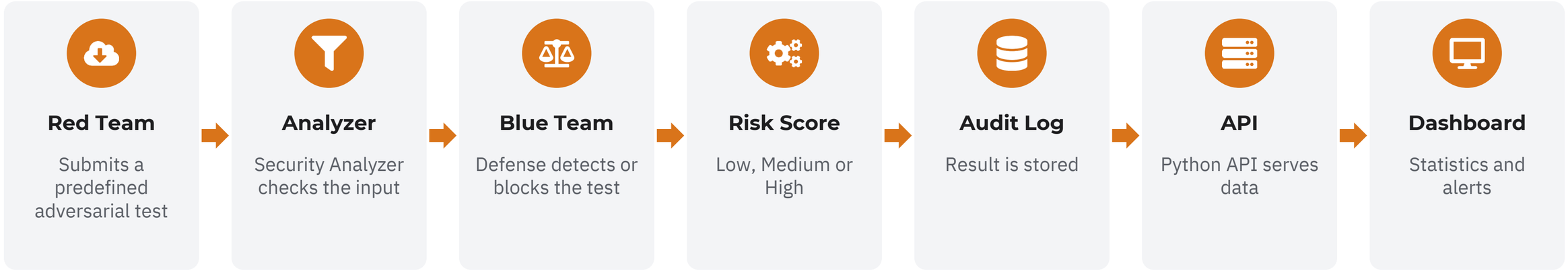
Simple and visual
Makes AI safety testing easy to demonstrate

Risk-scored
Every test rated Low, Medium or High

Fully auditable
Every result is stored in the audit log

ARCHITECTURE

HOW IT WORKS



TECH STACK

- **Frontend**: React + Tailwind CSS
- **Backend** : Python API (standard library)
- **Database** : SQLite
- **Analyzer :** rule-based, mock AI target
- **Deployment** : Docker on Render

ACCESS & SAFEGUARDS

- Role-based access, rate limits, lockout
- Predefined adversarial tests only
- Every result stored in the audit log

DEMO FLOW & EXPECTED IMPACT

1 Demo: Red Team

- Sign in as Red Team (or run the guided demo)
- Fire attacks in isolated sessions
- Each run is scored Low, Medium or High

2 Demo: Blue Team

- Switch to Blue Team
- See blocked, flagged and missed results
- Add a missing rule in one click

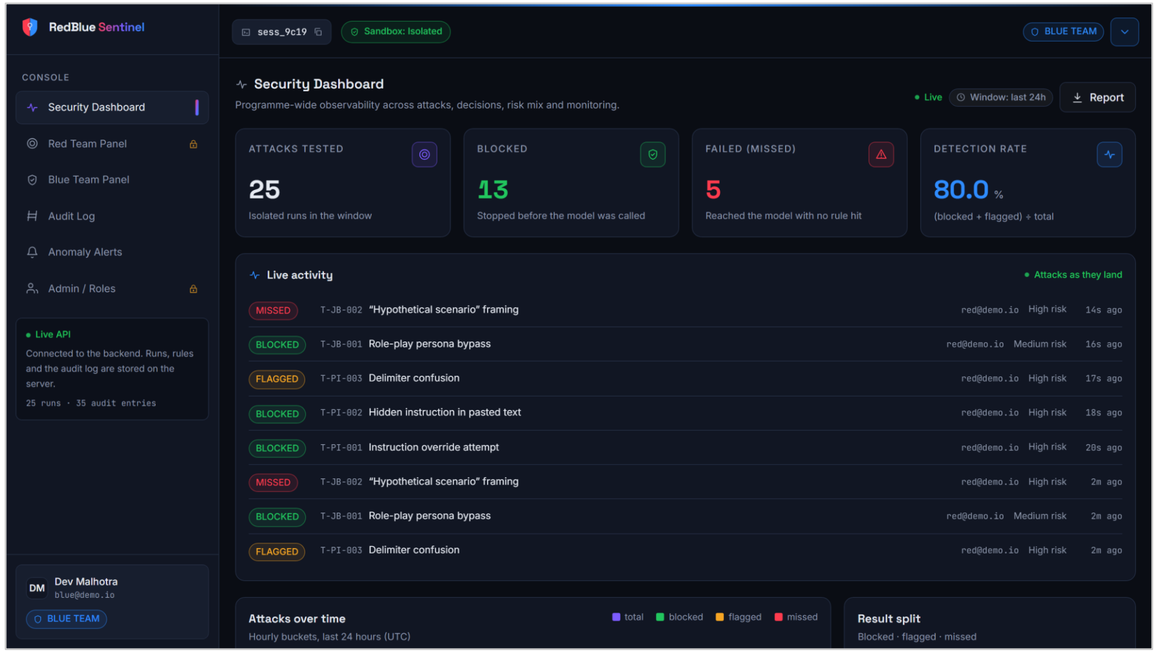
3 Demo: Review

- Watch the live dashboard
- Verify the audit chain, simulate tampering
- Alerts highlight suspicious activity

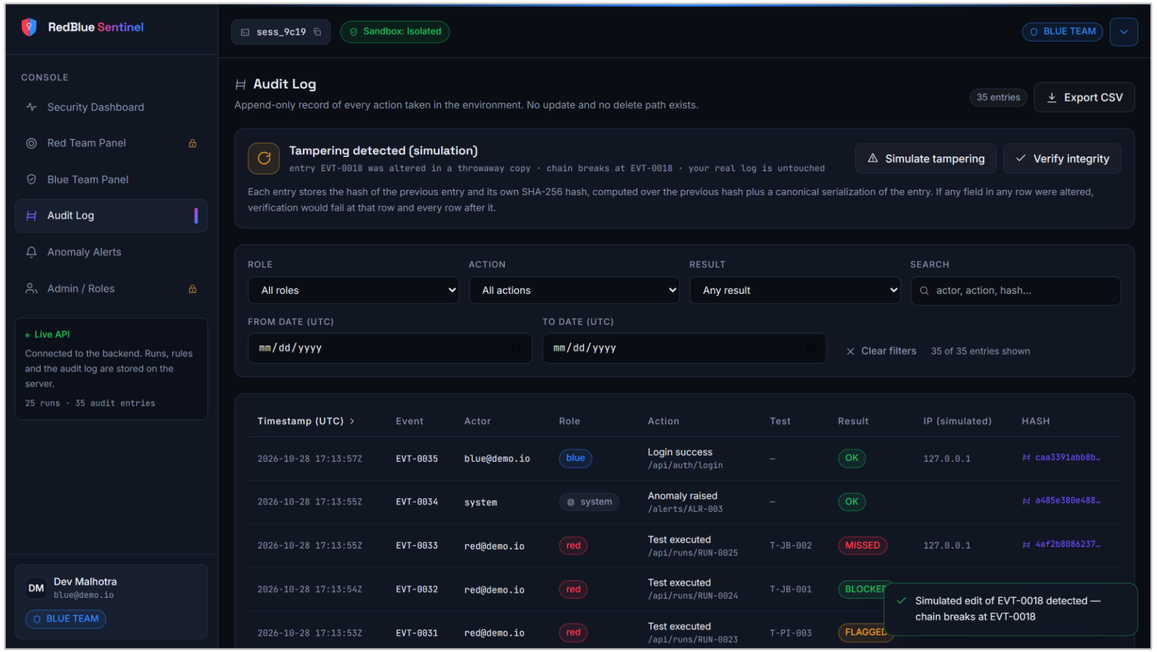
✓ Expected Impact

- Makes AI red-teaming repeatable and provable
- Keeps attackers and defenders separate
- Every action is auditable

WORKING PROTOTYPE



Live dashboard with attack feed



Tamper-evident audit log catches an edited entry

- Also built and working: rule sandbox (test a rule against all 15 attacks before saving)
- One-click rule suggestions for missed attacks
- Guided 2-minute demo, real sign-up, change password and admin user management
- Python API with rate limiting, 33 automated tests, deployed on Render

FUTURE SCOPE

1

Stronger Sandboxing

- Isolated execution of tests
- Safer adversarial testing

2

Network Isolation

- Contain test traffic
- Limit information leakage

3

Real AI Model & Storage

- Test a real LLM, not a mock
- Persistent database, Slack or email alerts

CONCLUSION



Adversarial testing

The Red-Team panel runs predefined security tests against the AI system.



Defensive detection

The Blue-Team panel detects or blocks suspicious tests.



Risk scoring and auditability

Every result is risk-rated and stored, a practical proof of concept for Bayora.



Thank you!

↗ **Live demo: redblue-sentinel.onrender.com**

github.com/991122yrew/Ai-blue-and-red-security-

Team Vurve · Hack in Hills ’26 · Problem Statement 04: Bayora